Noema × Tushe · Our Response to Your Draft

Responding to: *Tushe × Noema: Response to the Synthetic Enterprise Environment requirements*, Tushe engineering, 7 October 2026 **From:** Noema engineering **Date:** 12 October 2026 **Comes with:** *Acceptance Worksheet for Thursday 15 October*, which turns everything below into a test-by-test agreement

---

1. Thank you, and where we've landed

Thank you for a thorough and candid response. Several things stood out:

- you answered every section we sent;
- you named what isn't there yet instead of smoothing it over;
- where you think our specification is wrong, you said so, with sources.

That is exactly the working relationship we hoped for.

Our headline: we can work with what you have today, and we'd like to start this week. The capabilities we lean on hardest already exist and are running:

- a payout whose own record, the rail and the beneficiary can disagree, with nothing reconciling them;
- the clear line between "it failed" and "we don't know";
- the status query;
- seeded, checksummed, resettable scenarios with predictable IDs;
- a replayable event feed with a sequence cursor;
- deterministic idempotency conflicts;
- read-only credentials.

On your challenges: you are right on most of them, and we've changed our approach. Where we accept your alternative, we've written down the exact condition so neither of us has to argue about it on Thursday.

We're also taking work off your list. Several items we asked for are things we can do better on our side. Section 5 lists them, so your time goes where only you can help.

2. Our answers to your six questions (your §10)

1. Cases and disputes: out of scope for you. You're right that case handling sits closer to our product than to payments infrastructure. For this POC we will run our own small synthetic case system, and Noema's writes go there. Disputes are not part of the POC.

A useful side effect: our credentials against your sandbox can stay **read-only for the entire POC**.

2. GT-02, the privileged answer key: we accept your counter-proposal. Your reasoning in §5.2 is sound. We'll establish what actually happened from three things:

- the checksummed scenario definition;
- your partner-administrator feed (`GET /partner/events`);

- the transition log, once its read endpoint ships.

We need two things for this to work:

- **a programmatic, read-only administrator credential** our test harness can use without a browser session, issued to one named person on our side and never to the system under test;
- **the refusal test** proving an ordinary API key cannot reach `/partner/...` routes.

If either can't be provided, we'd come back to GT-02 as written. We don't expect to need to.

3. Scenario 19, the impossible state: we accept your replacement. You shouldn't switch off your invariants or fake state at your read layer. A cross-system contradiction is the better test, and it's achievable today. Where we want to see how our own software handles data that can't be true, we'll produce it on our side and label it as such.

4. "Material": we accept your definition, with one addition. Your §5.4 wording plus this: *for each scenario object, the ordered list of lifecycle transitions (from-state, to-state, source) also reproduces exactly.* Wall-clock timings and ordering between tenants are excluded, as you proposed.

5. Concurrency: one tenant per concurrent run works for us. We expect to need about ten test tenants across developers, CI, nightly runs and demos. Can tenants be created through an API, or only by your team?

6. The P1s: none, apart from one item we think is really a P0. - EVT-05 (signing) is already done. - We don't need GT-05 (export): we'll assemble run records from your read surfaces. - We don't need CON-01 (version tokens): our write target is now our own case system.

The one item we do need is **switch state as its own queryable dimension**, which you described as an additive change under SE-01. Our specification lists the switch as an independent system (SE-01, P0), and several of our contradiction cases turn on the switch saying one thing while the beneficiary side says another.

3. Your challenges, one by one (your §5 and §6)

The fast-forward clock (§5.1): you're right

Our TIM-02 contradicts our own DATA-05. We accept per-object scheduling with on-demand advance. We'll handle the rest of our time-sensitive testing on our side.

We are withdrawing the request for a global clock. AT-15's pass condition becomes: *"a scheduled transition fires at its scheduled time, and fires immediately on manual advance."*

The answer key (§5.2): agreed, with the conditions above

Grading only the final state (§5.3): agreed, and no change needed from you

You're right that final state alone can't tell "correctly escalated" apart from "did nothing". Our grading already looks at the process:

- which systems were asked;
- whether anything was re-sent while the outcome was unknown;
- whether the right records were written.

We also repeat runs to measure reliability rather than one lucky pass. Your transition log, event feed and audit trail are exactly the inputs that makes possible, so nothing changes on your side for this.

"Material" (§5.4): agreed, as in section 2

Scenario 19 (§5.5): agreed, as in section 2

Buying a proxy rather than building transport faults (§6): agreed in principle, and we'll handle it

We agree that deliberately broken code shouldn't live in your money path. Rather than either of us buying a product, we'll run a thin proxy of our own in front of the sandbox. It will produce:

- timeouts, resets and gateway errors;
- slow responses;
- stale reads;
- event delivery faults on the path where we read your feed.

You don't need to build or buy anything for this. Commit-then-no-answer stays yours (test account `02`), for exactly the reason you gave.

Transaction Status Query (your §4.6): thank you

This is a genuinely useful addition, and we'll use it as a read when an outcome is unknown. We'd like written confirmation of one thing: **calling it can never move money, reverse a debit or change a balance.** It may move a payout from unknown to a final status, but nothing more.

4. A few corrections, so the document stays a reliable record

You said the draft is a working document and you'd keep refining it. These are the places where it disagrees with itself or with our specification. None of them changes our decision to proceed.

1. **The acceptance-test tally.**
   - The summary in §8 says **12 pass, 6 partial, 2 fail**.
   - The table above it shows **11 pass** (counting AT-15's "pass, differently"), **4 partial** (AT-01, 05, 14, 19) and **5 fail** (AT-04, 07, 09, 17, 20).
   - The table is the one we're working from.
2. **The scenario tally.**
   - The summary says **11 executable, 5 partial, 3 requiring extension, 1 challenged**.
   - The table shows **12 executable**: 1, 2, 3, 4, 5, 6, 9, 10, 12, 13, 14, 16.
   - It shows **4 partial**: 7, 15, 17, 20.
3. **Requirements not covered.**
   - Our specification has **85 numbered requirements (73 P0, 12 P1)**. Your response covers 73 (63 P0, 10 P1).
   - The difference is exactly our ledger requirements, **LED-01 to LED-06 (§4.3)**, which have no rows.
   - Our six idempotency requirements, **IDP-01 to IDP-06 (§11)**, are answered in one combined row.

- Could you add rows for all twelve? Three matter most to us:
  - LED-04: one leg posted, another delayed or uncertain;
  - LED-05: integer minor units;
  - IDP-06: idempotency-key retention.

4. **"The only remaining item your acceptance tests depend on" (§1, §4.7).**
   - That describes event-delivery faults (E3).
   - Your own table also ties AT-09 and AT-14 to E6, AT-17 to E7, AT-19 to E1, and AT-20 to E5.
   - With our proposed exceptions, this stops mattering, but the sentence should match the table.
5. **SE-01 is marked "Supported".** Your note says the switch is not a separately queryable state. We'd describe that as "Supported differently" until switch state is queryable on its own.
6. **TIM-03 says `delivered` doesn't exist because outbound delivery (E2) doesn't exist.** E2 is now marked built, and `deliveredAt` exists, per EVT-01. The TIM-03 row looks out of date.
7. **Scenarios 6 and 7.**
   - Scenario 6 (missing webhook) is mapped to a payout that never resolves. That's a different case: in a missing webhook, the state *does* change but the event never arrives.
   - Scenario 7 (out-of-order webhook) is mapped to reversal-after-completion, which is real causal order rather than out-of-order delivery.
   - Both are fine to keep as their own cases. We'll produce the delivery-side versions on our side.
8. **The indicative dates in §11 (24 Oct, 7 Nov, 21 Nov, 28 Nov, 12 Dec) all fall on Saturdays.** Working-day dates would make planning easier on both sides.
9. **Our mistake: your name.** Our specification spells your name "Tuche" throughout. We apologise; it will be correct in our next revision.

5. What you can take off our list

So your effort goes where only you can help, **none of these is on our critical path for the POC**:

- **Deliberate event drop, delay, reorder and malformed delivery (E3).** We produce these on our side.
- **The diagnostics plane (E5) and run export (E7).** We assemble what we need from your read surfaces.
- **Generic version tokens and the stale-read profile (E6).** Not needed: our write target is our own case system, and stale reads come from our proxy.
- **Ambiguous name enquiry (E6, AT-09).** Welcome later; not needed for our first demo.
- **A case or dispute domain.** Out of scope for you.
- **A privileged oracle endpoint (GT-02).** Replaced by your administrator surface, on the conditions in section 2.

6. What we need from you

In priority order. The worksheet has the same list with a reason for each.

#	Request	Date we're asking for
R1	About ten test tenants; a read-only API key per tenant; one programmatic read-only administrator credential; one test-mode credential that can create payouts for building scenario histories (held by one named person on our side, never by the system under test)	**Wed 14 Oct**
R2	The transition-history read endpoint (STM-03) on its own, ahead of the rest of E1	**Mon 19 Oct**
R3	Scenario definitions that seed past payouts with set beneficiary names, amounts, outcomes and `occurredAt` dates, including near-duplicates (a small subset of E4)	**Fri 23 Oct**
R4	Switch state as its own queryable dimension: raw code, normalised status, and whether it is final	**Mon 19 Oct** if possible; otherwise your earliest date
R5	No behaviour change on the sandbox from 13 Oct to 6 Nov without 48 hours' notice, and the sandbox version on every response	From **Tue 13 Oct**
R6	Written confirmation that the status query can never move money or change balances	**Tue 13 Oct**
R7	Test-mode rate limits and quotas, and the sandbox's hosting region	**Tue 13 Oct**

And six short questions:

1. Does reset also remove payouts created through the API on a scenario's accounts?
2. Do API-created payouts in a freshly applied scenario get the same IDs every time, given the same idempotency key and reference?
3. Can test tenants be created through an API?
4. Are webhooks optional for us? We plan to read `GET /v1/events` by sequence instead.
5. Are amounts integer kobo everywhere, including in events?
6. What availability should we expect between 13 October and 6 November, and who do we contact when the sandbox is down?

7. What Noema commits to

- **Our own synthetic case system**, so you never have to model case handling.
- **Our own proxy** for transport, stale-read and event-delivery faults, so no deliberately broken code goes near your money path.
- **Reading your feed by sequence** rather than relying on webhook delivery.
- **Read-only use of your sandbox for the whole POC.** The one payout-creating credential is used only to build test scenarios, by one named person.

- **Showing you the tests we cover ourselves** (AT-04, 05, 07 and 14) by Friday 16 October, and sharing the evidence.
- **Reporting anything that looks like a sandbox defect** with the tenant, sandbox version, scenario, seed and the redacted request and response, so you can reproduce it before anyone fixes anything.

8. Next steps

When	What
Tue 13 Oct (one hour, at a time that suits you)	Walk through and sign the acceptance worksheet; answer the questions in section 6; confirm R1 to R7
Wed 14 Oct	Tenants and credentials issued (R1). Our team starts integrating
Thu 15 Oct, 10:00–12:00 WAT	Joint acceptance session, run against the signed worksheet. Both sides sign the record
Fri 16 Oct	Noema shows the tests it covers on its side
Mon 19 Oct	Transition-history endpoint (R2) and, if possible, switch state (R4)
Fri 23 Oct	Seeded histories (R3)
Every Friday until 6 Nov	A 30-minute check-in: what landed, what slipped, what's next

Thank you again. It's clear your team took our specification seriously, and we've tried to answer in the same spirit. We're looking forward to Tuesday.

Noema engineering